PSOC™ Edge Encryption training manual

About this document

Scope and purpose

This document is a training manual for the hands-on exercise of encrypted boot using Edge Protect Bootloader on PSOC™ Edge using ModusToolbox™.

Intended audience

The intended audiences for this document are design engineers, technicians, and developers of electronic systems.

Table of contents

About this document **1**
Table of contents **2**
1 Introduction **3**
2 Required development tools **4**
3 Prerequisites **5**
4 Single-key XIP encryption with EPB **6**
4.1 Objective 6
4.2 Description 6
4.3 Project creation 7
4.4 Encrypted Boot terminal output in KIT_PSE84_EVAL_EPC2 15
4.5 Encrypted Boot terminal output in KIT_PSE84_AI 16
4.5.1 Provisioning method 16
4.5.2 Hardware rework 16
4.5.3 Observe the output 17
4.6 Conclusion 17
5 Multi-key XIP encryption **18**
5.1.1 Project creation 18
5.1.2 Enable multikey encryption 18
5.2 Conclusion 21
Revision history **22**
Disclaimer **23**

1 Introduction

This manual provides specific instructions to create, configure, build, and run the code examples on the PSOC™ Edge E84. In this lab session you will learn how to use Edge Protect Bootloader (EPB) on PSOC™ Edge to boot an encrypted application.

2 Required development tools

The hands-on labs require either KIT_PSE84_EVAL or KIT_PSE84_AI

- ModusToolbox™ software v3.8 or later
  - Recommended installation via ModusToolbox™ Setup tool
- Eclipse IDE for ModusToolbox™ v2026.3.0 or later
  - Recommended installation via ModusToolbox™ Setup tool
- Edge Protect Security Suite v1.6.1 or later
  - Installed by ModusToolbox™ Setup tool as a dependency to ModusToolbox™
- ModusToolbox™ Programming Tools v1.8.1 or later
  - Installed by ModusToolbox™ Setup tool as a dependency to ModusToolbox™
- Board support package (BSP)
  - KIT_PSE84_EVAL_EPC2: v1.3.0 or later (available with ModusToolbox™)
  - KIT_PSE84_AI: v1.3.0 or later (available with ModusToolbox™)
- Terminal emulator
  - Instructions in this document use Tera Term
- PSOC™ Edge E84 Evaluation Kit (KIT_PSE84_EVAL)
  - Ensure device boots from external memory
    - BOOT SW/SW6 in 'OFF/LOW' position
  - Disable alternate serial interface configuration
    - J20/J21 in Tristate/Not-Connected (NC) position
- PSOC™ Edge E84 AI Evaluation Kit (KIT_PSE84_AI)

Note: This training manual was created using the tools above, but other versions of tools and PSOC™ Edge kits are expected to work with minimal modifications.

3 Prerequisites

Install the software and obtain the hardware listed in the Required development tools section.

This class will not cover basic concepts of ModusToolbox™ and PSOC™ Edge.

- For an introduction to PSOC™, including a getting started guide to ModusToolbox™, visit:

https://www.infineon.com/product-information/psocdeveloper

- For PSOC™ Edge trainings, from beginner tutorials to advanced trainings, visit:

PSOC™ Edge E84 Training Collection

4 Single-key XIP encryption with EPB

4.1 Objective

In this lab, we will use the Encrypted Boot code example to demonstrate the XIP encryption capabilities of the Edge Protect Bootloader (EPB).

4.2 Description

Firmware images may be encrypted and programmed into the device memory. Edge Protect Bootloader supports the encrypted firmware when EPB runs from RRAM only. Encrypted user images must be located in external flash memory and either executed in SMIF XIP mode or loaded into SRAM for execution.

Edge Protect Bootloader supports the execution of the encrypted application in the following methods:

- Single-key encryption (EPC 2)
- Multi-key encryption (EPC 2)
- Secure Encryption (EPC 4)

This module focuses on using the Encrypted Boot code example for the single-key and multi-key encryption use case for the EPC 2 devices.

4.3 Project creation

1. Before creating the first project, we need to create our workspace in ModusToolbox™. Open **ModusToolbox™** from the **Windows Start** menu

Note: *If you have not installed ModusToolbox™, see the Required development tools section. If you have already created a workspace, please ignore this step and move on to step 4 directly.*

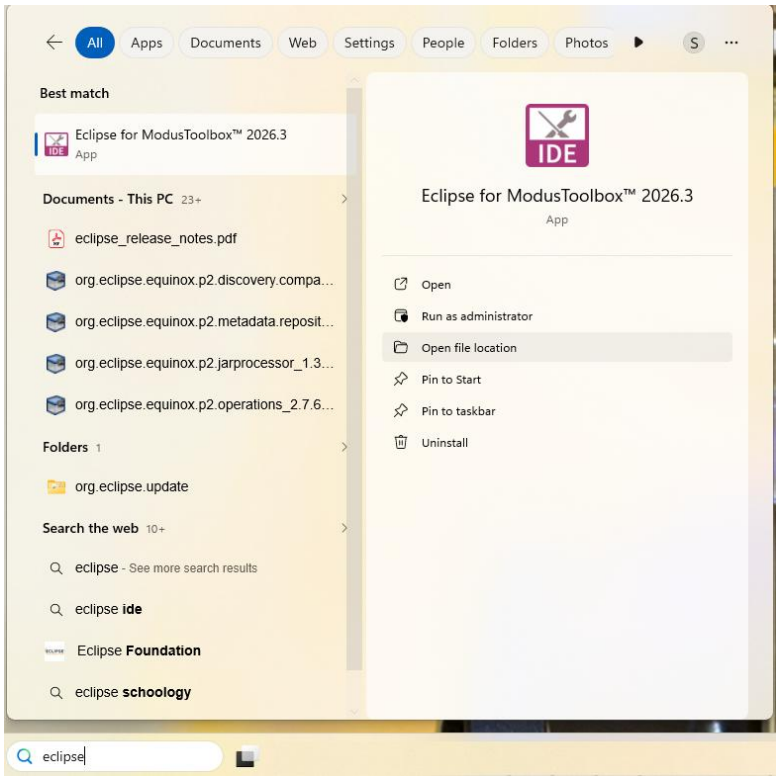


Figure 1 Start ModusToolbox™

2. **Next**, choose a **workspace directory** for your project. After selecting a suitable directory andclick **Launch**

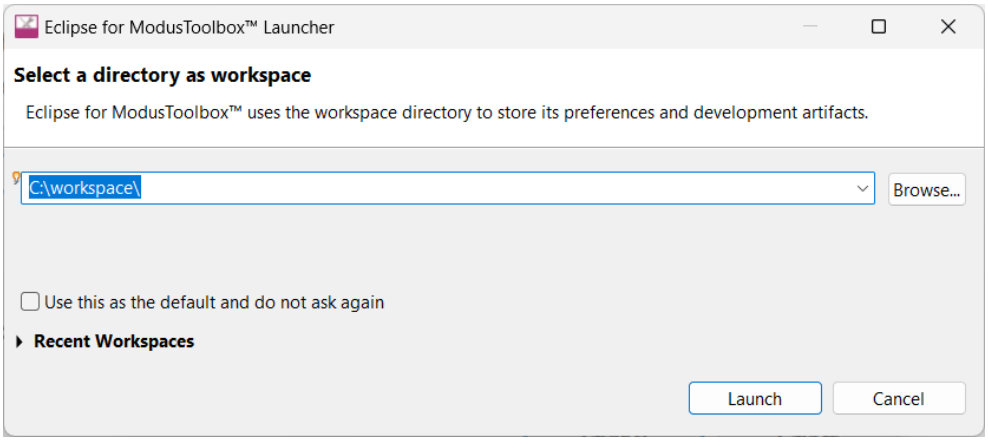


Figure 2 Select workspace directory

3. **Launch** opens a new ModusToolbox™ workspace

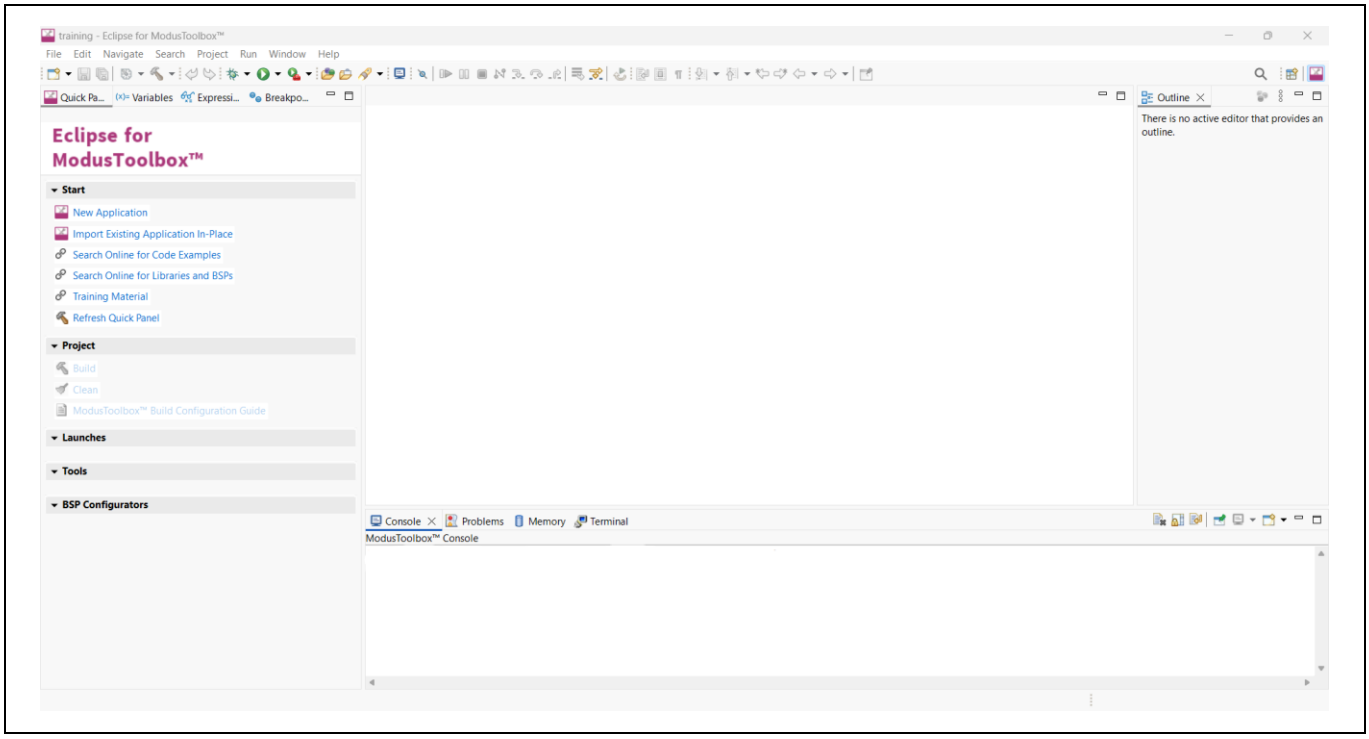


Figure 3 Launch ModusToolbox™ workspace

4. Now select **New Application** from the **Quick Panel**. Alternatively, go to **File** > **New** > **ModusToolbox™** application

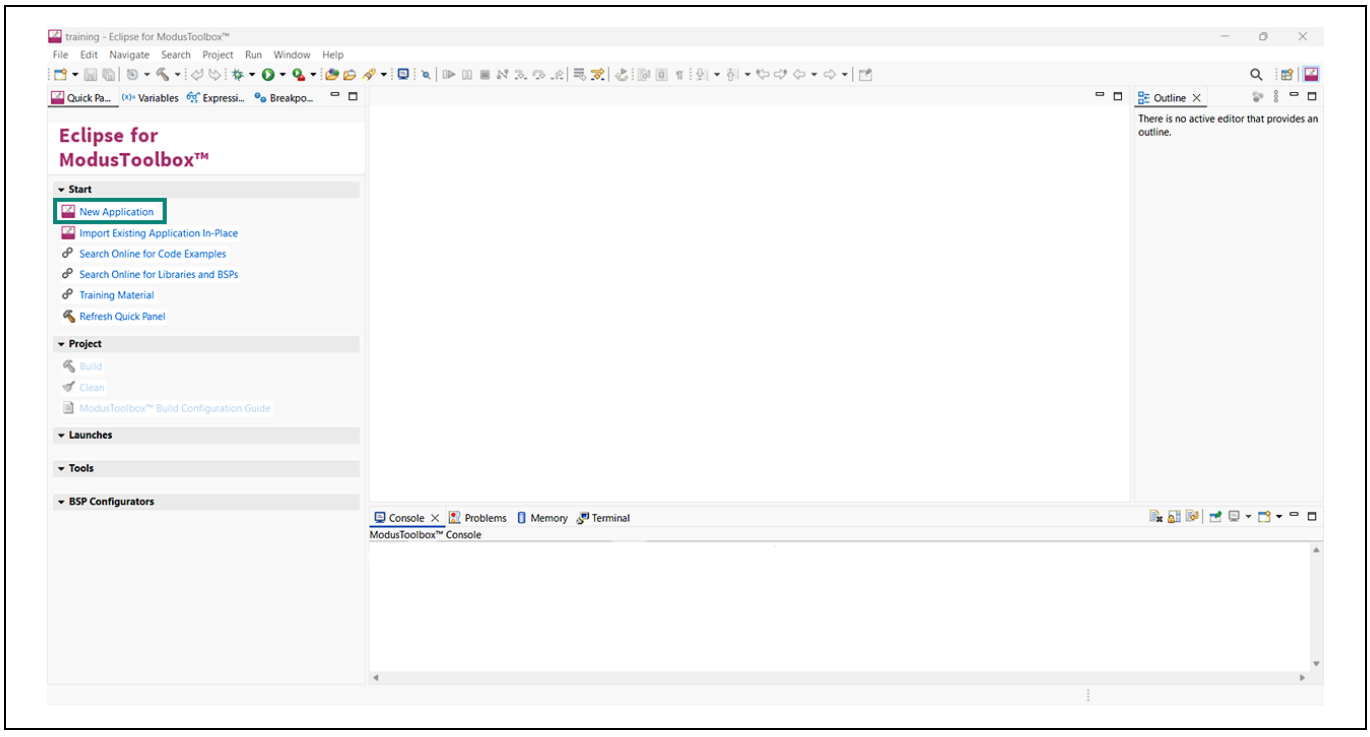


Figure 4 Create new application

5. Clicking New Application opens the **Project Creator** and prompts you to choose a **board support package** (BSP)

Note: *BSPs are aligned with our development/evaluation kits; they provide files for basic device functionality. A BSP typically has a **design.modus** file that configures clocks and other board-specific capabilities. That file is used by the ModusToolbox™ configurators. A BSP also includes the required device support code for the device on the board. You can modify the configuration to suit your application.*

6. Once **Projector Creator** launches, select either the **KIT_PSE84_EVAL_EPC2 / KIT_PSE84_AI BSP** under the PSOC™ Edge BSPs, and click **Next**

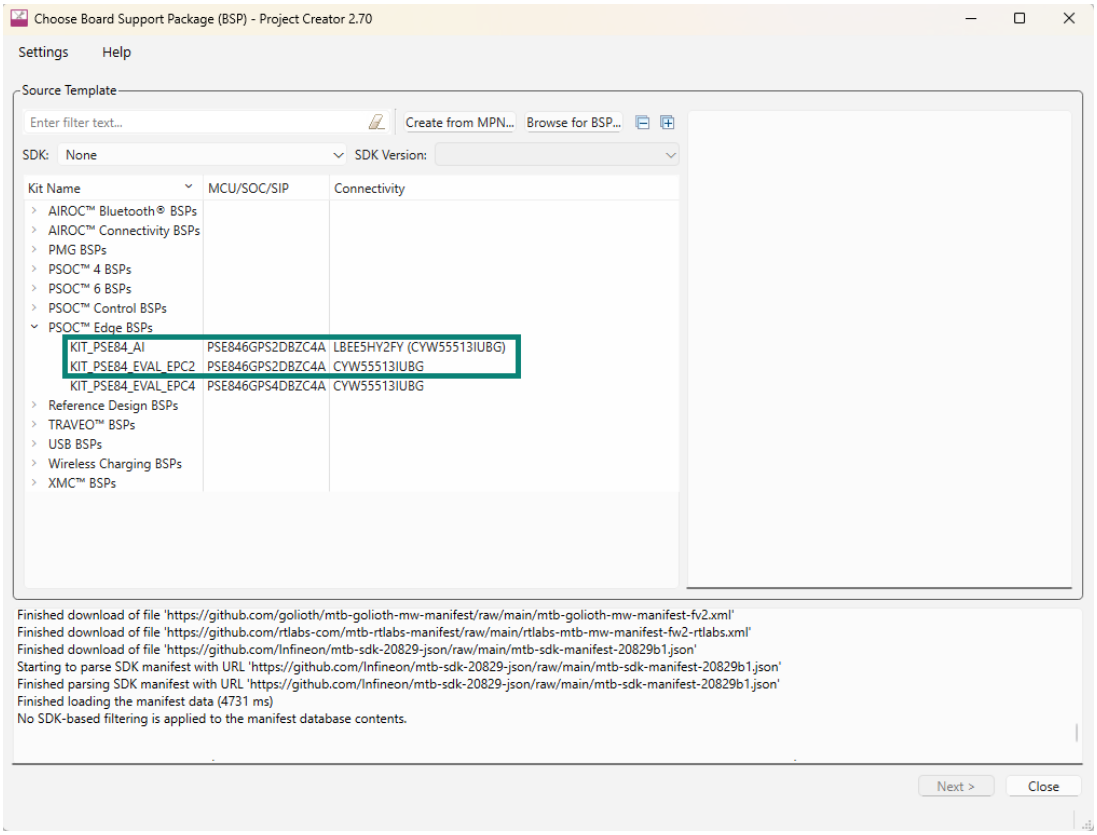


Figure 5 Select BSP in project creator

7. Clicking **Next** takes you to the next menu of the **Project Creator**, which asks you to select an application:
   a) Select the **PSOC™ Edge Protect Bootloader** under the **Security** section. You can do this by selecting the **checkbox** near the application
   b) Select the **PSOC™ Edge Encrypted Boot** under the **Security** section. You can do this by selecting the **checkbox** near the application
   c) Click **Create**

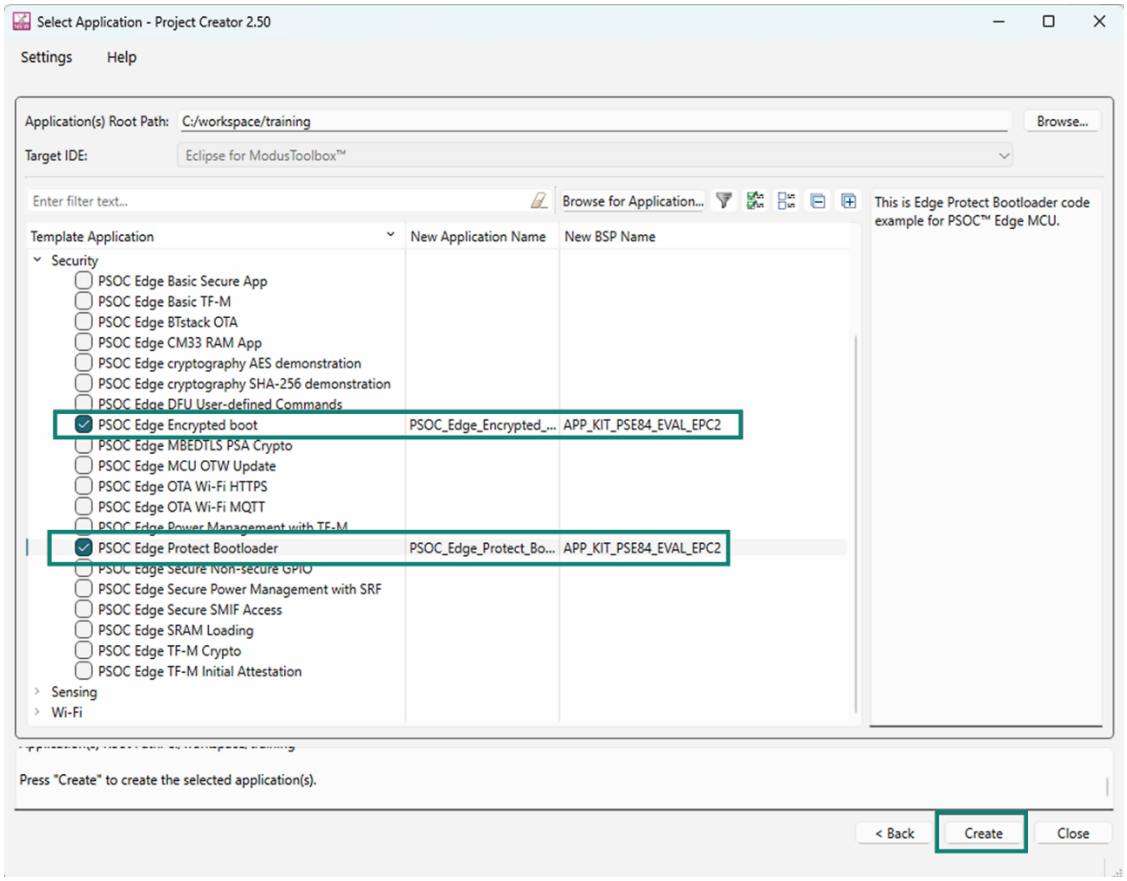


Figure 6 **Select application in project creator**

Note: *Complete demonstration of Edge Protect Bootloader requires Basic Secure App. Creating both code examples together in the above step.*

8. When you click **Create**, two new ModusToolbox™ applications are created. You can see both applications in the **Project Explorer** window

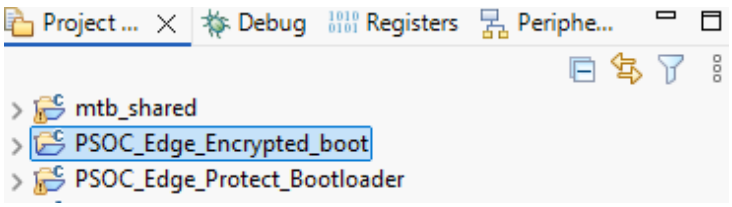


Figure 7 Project Explorer window

9. Right-click and copy **proj_bootloader** under Edge Protect Bootloader

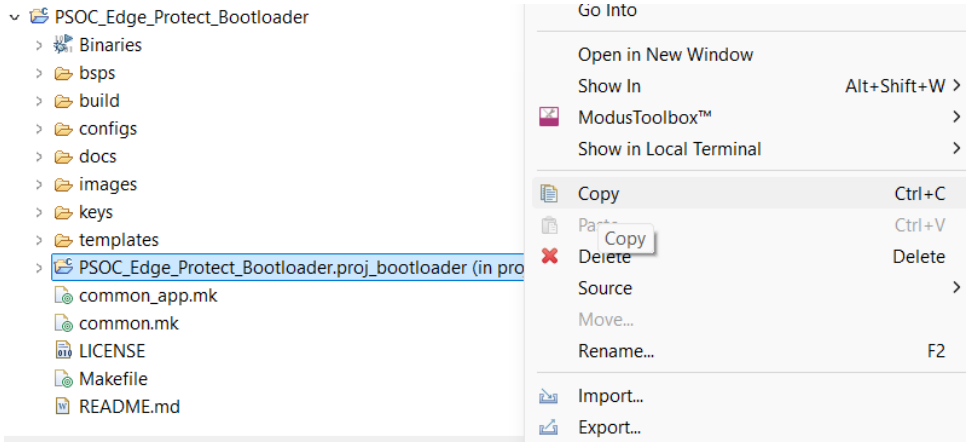


Figure 8 Copy proj_bootloader

10. Right-click and paste the **proj_bootloader** to the root of Encrypted Boot

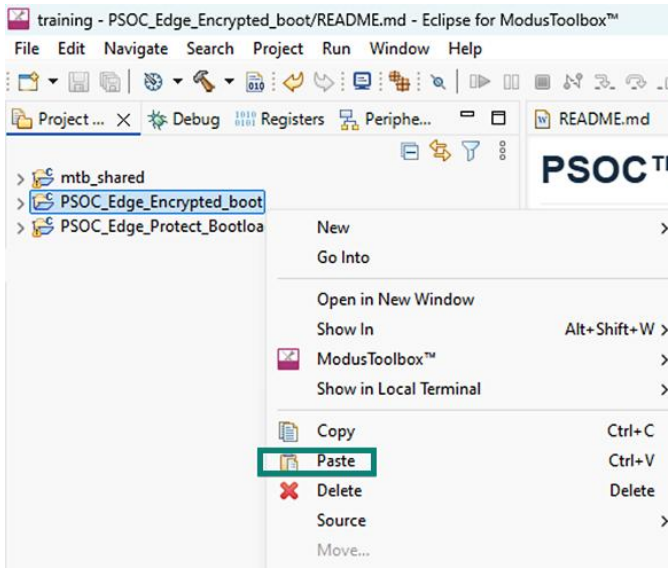


Figure 9 Paste proj_bootloader

11. While pasting, edit the project name and location to point to the Encrypted Boot code example

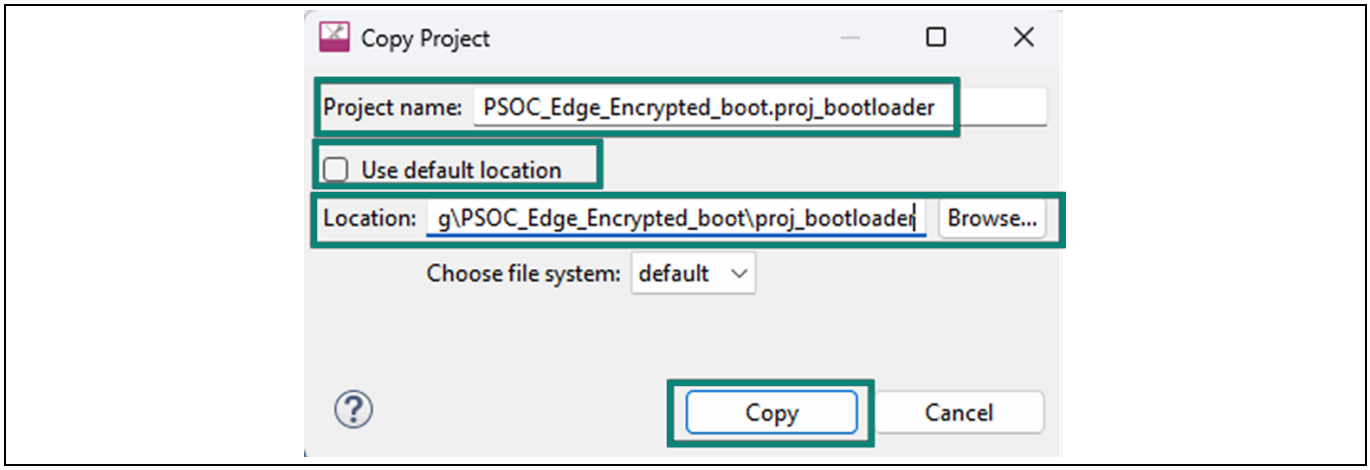


Figure 10 Edit the project name and location

12. Add the **proj_bootloader** to the list of projects in the makefile of the Encrypted Boot code example

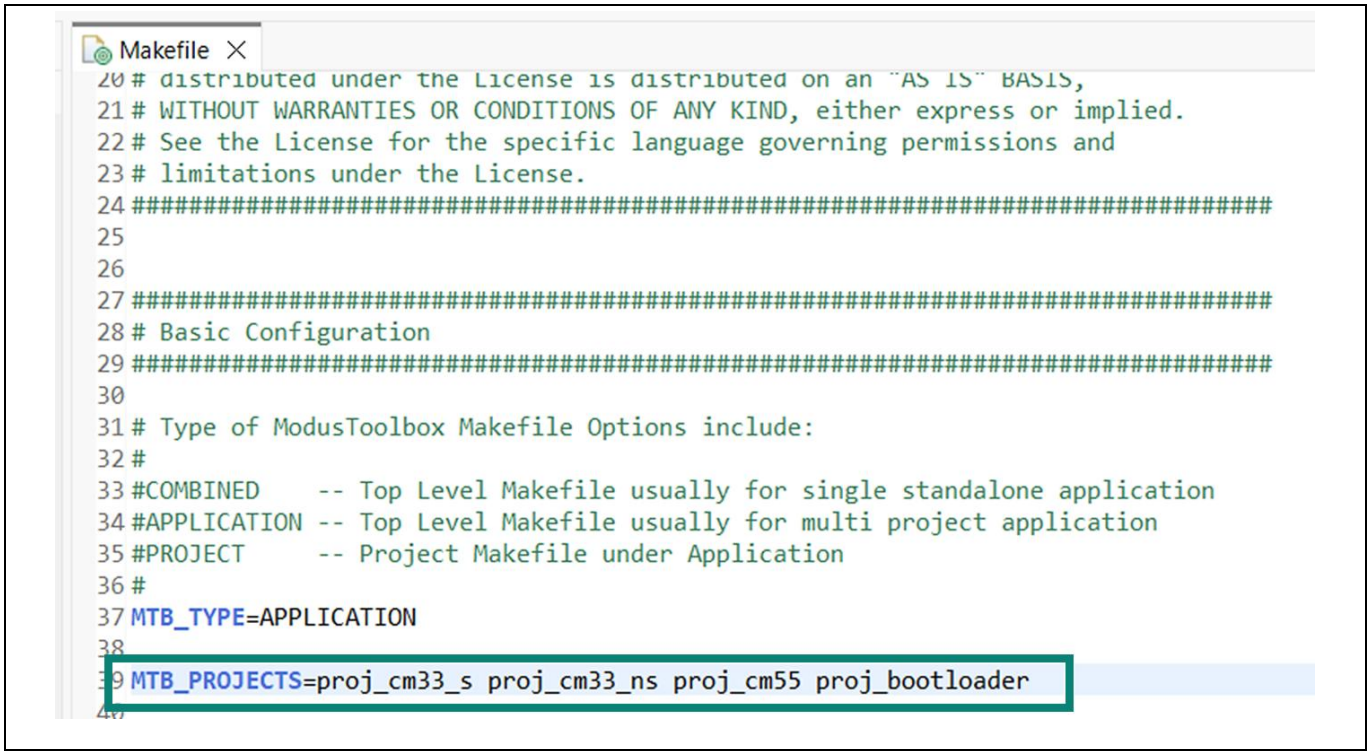


Figure 11 Add proj_bootloader to the ModusToolbox™ projects list

13. Update the *common.mk* file of Encrypted Boot to point to the bootloader personality generated JSON file

common.mk

```
46 #CY_COMPILER_IAR_DIR=C:/Program Files/IAR Systems/Embedded Workbench 9.2/arm
47 #CY_COMPILER_ARM_DIR=C:/Program Files/ArmCompilerforEmbedded6.22
48 #CY_COMPILER_LLVM_ARM_DIR=C:/Program Files/LLVM-ET-Arm-19.1.5-Windows-x86_64
49
50 # Default build configuration. Options include:
51 #
52 # Debug -- build with minimal optimizations, focus on debugging.
53 # Release -- build with full optimizations
54 # Custom -- build with custom configuration, set the optimization flag in CFLAGS
55 #
56 # If CONFIG is manually edited, ensure to update or regenerate
57 # launch configurations for your IDE.
58 CONFIG?=Debug
59
60 #Config file for postbuild sign and merge operations.
61 #NOTE:Check the JSON file for the command parameters
62 COMBINE_SIGN_JSON?=./bsps/TARGET_$(TARGET)/config/GeneratedSource/boot_with_bldr.json
63
```

Figure 12 Update *common.mk* file to point to the bootloader personality generated JSON file

14. Generate launches for the Encrypted Boot code example

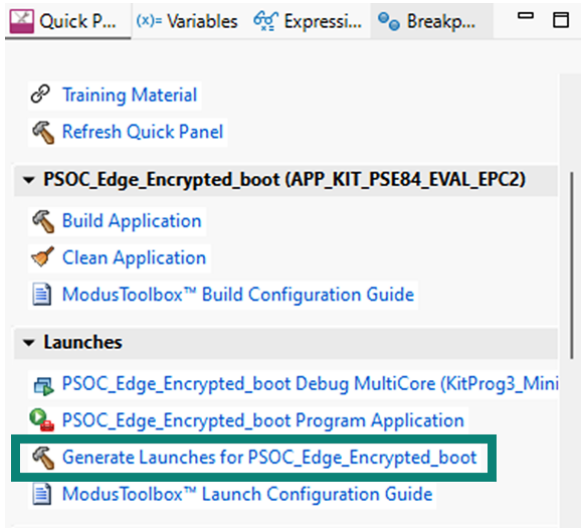


Figure 13 Generate Launches

15. Follow the steps in the Edge Protect Bootloader lab module to enable Secure boot flow for the EPB-based Encrypted Boot application

16. Generate the AES-128 Key for image encryption using the following command

```
edgeprotecttools create-key --key-type AES128 --output keys/aes128_key.bin
```

17. Generate the key encryption key (KEK) using the following command

```
edgeprotecttools create-key --key-type ECDSA-P256 --output keys/enc-ec256-priv.pem keys/enc-ec256-pub.pem
```

18. Convert the PEM private key to the DER-PKCS8 format using the following command

```
edgeprotecttools convert-key -k keys/enc-ec256-priv.pem -o keys/enc-ec256-priv.bin -f DER-PKCS8
```

19. Program the private portion of the Key-Encryption-Key (KEK) to the device using openocd at the start address of the “kek_region” defined in the memory map. Check your RRAM memory map in **Device Configurator** for the correct location of the **kek_region**. The default configuration places **kek_region** in **0x32039000**

```
<openocd-path>/openocd.exe -s scripts -f interface/kitprog3.cfg -f target/infineon/pse84xgxs2.cfg -c "init; reset init; flash write_image keys/enc-ec256-priv.bin 0x32039000; reset; shutdown"
```

20. Now build and program the Encrypted Boot application by clicking the **Program Application** in the **Quick Panel**

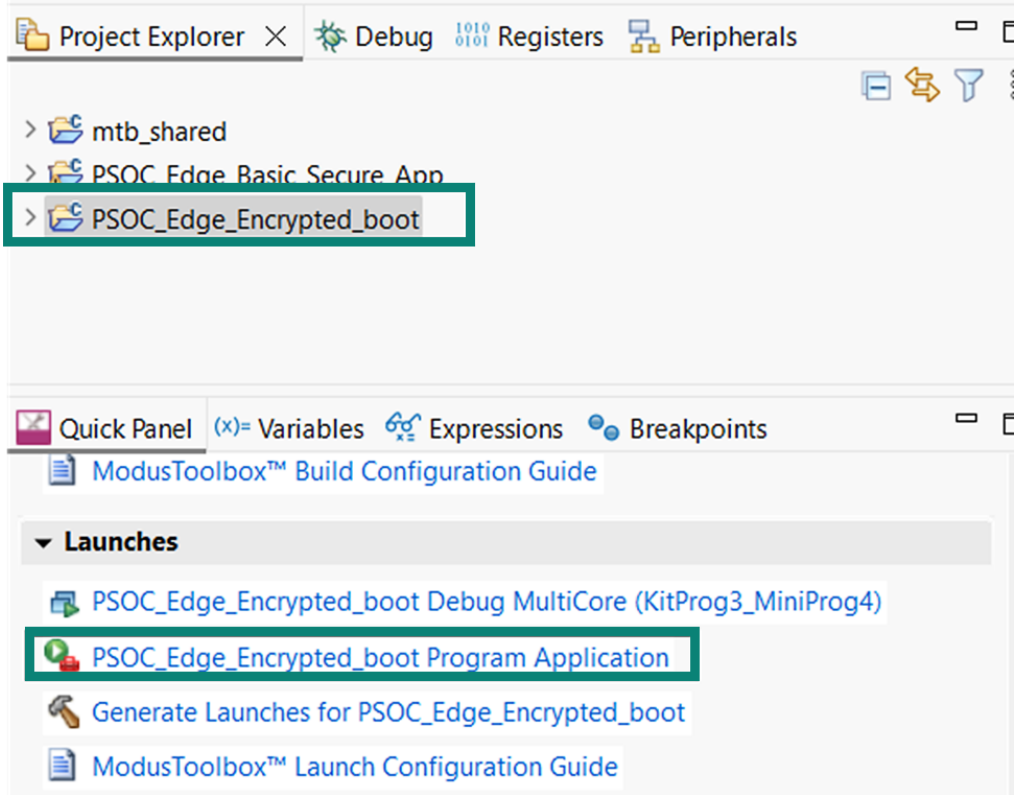


Figure 14 Program application

4.4 Encrypted Boot terminal output in KIT_PSE84_EVAL_EPC2

1. Set the Boot SW (SW6)/P17.6 position to “LOW/OFF”. This will instruct Extended boot to boot the first application from RRAM. Edge Protect Bootloader is the first application in this case, which is built and programmed to RRAM in the default case
2. Reset/power cycle the device
3. Observe the Tera Term console logs for Encrypted Boot

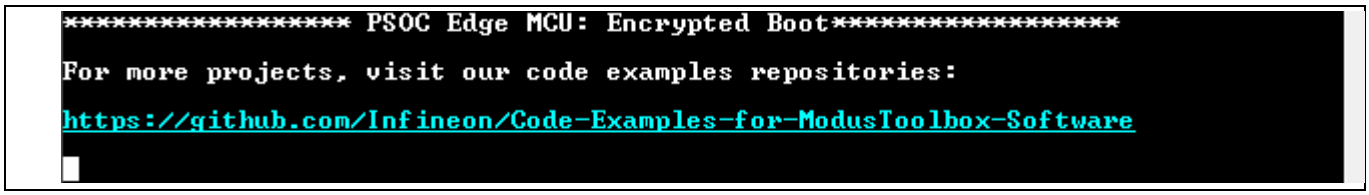


Figure 15 Terminal output for Encrypted Boot

4. Scroll up to observe the crypto services being initialised and utilized by Edge Protect Bootloader

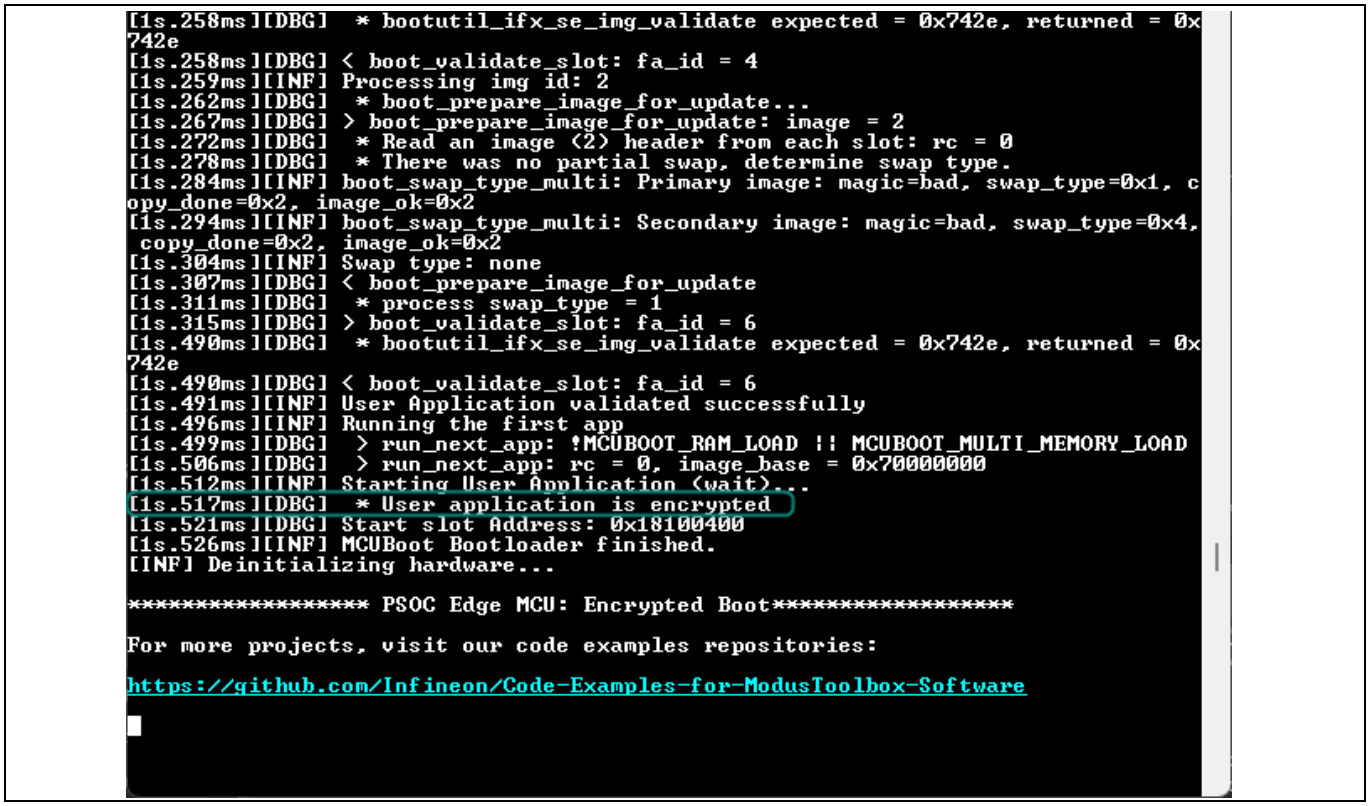


Figure 16 Edge Protect Bootloader output in the terminal

4.5 Encrypted Boot terminal output in KIT_PSE84_AI

Unlike the KIT_PSE84_EVAL_EPC2, the boot pin in KIT_PSE84_AI is not configurable through a slide switch. In this kit, the boot pin is configured with a default pull-up, enabling flash boot by default.. To boot from RRAM, follow one of the methods below.

4.5.1 Provisioning method

The process for transferring device ownership is detailed in the **Training Manual PSOC™ Edge E84 Getting Started with Security**. Ensure to follow the instructions outlined in the manual to generate the key pair and provision the device using edgeprotecttools to obtain device ownership. If you have previously obtained device ownership through this process, you can skip this step.

It is important to note that the same key pair (OEM_RoT_Key) used for provisioning must also be used for signing and verification.

Edit *<application-directory>/policy/policy_oem_provisioning.json* file using any text editor to set the value field to false for the “oem_alt_boot” parameter. This bypasses the boot pin status check and instead boots from the “oem_app_address” mentioned in the policy file.

By default, the oem_app_address in the *policy_oem_provisioning.json* points to RRAM location.

```
"oem_alt_boot": {
  "description": "Disable/Enable alternate boot by setting the value to false/true",
  "value": false
},
```

Figure 17 Disable oem alt boot

4.5.2 Hardware rework

Do the below changes to the kit to pull the boot pin to the LOW (0) state:

- Remove the resistor R188
- Populate a 0 Ω resistor at R187

To pull the boot pin to the HIGH (1) state:

- Remove the resistor R187
- Populate a 0 Ω resistor at R188

4.5.3 Terminal output

1. Reset/power cycle the device
2. Observe the Tera Term console logs for Encrypted Boot

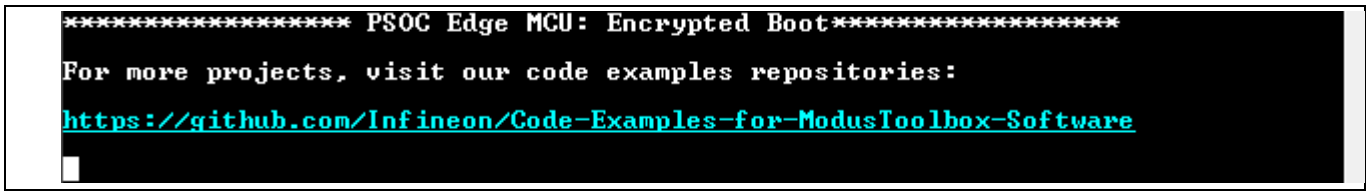


Figure 18 Terminal output for Encrypted Boot

3. Scroll up to observe the crypto services being initialised and utilised by Edge Protect Bootloader

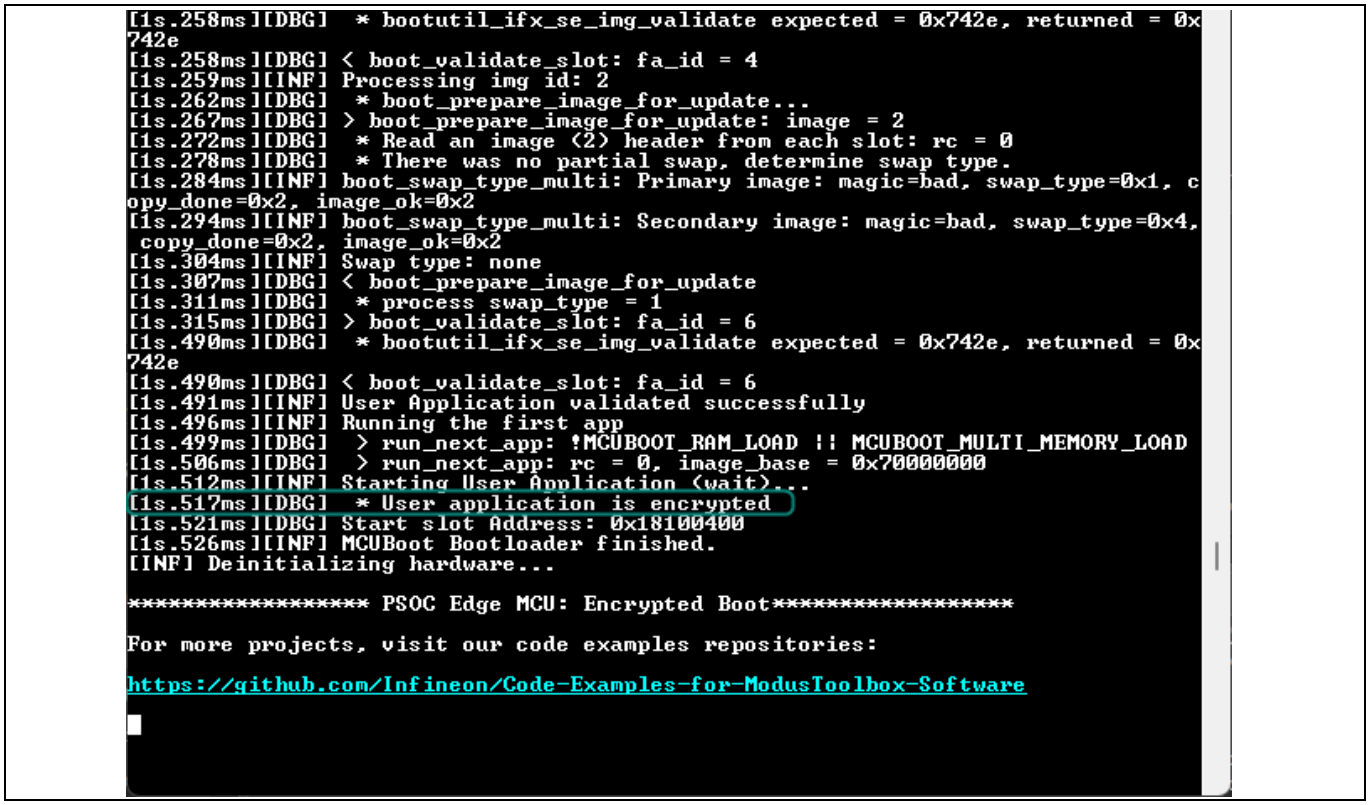


Figure 19 Edge Protect Bootloader output in the terminal

4.6 Conclusion

We successfully tested the single-key encryption using the Encrypted Boot code example and learned:

- How to create the Encrypted Boot code example
- How to add a bootloader to the code example and enable secure boot
- How to program and boot to the encrypted application

5 Multi-key XIP encryption

This is a do-it-yourself exercise.

However, you can see the following steps for the solution.

5.1.1 Project creation

We will use the same Encrypted Boot code example that was created in the previous module for this demonstration.

If not already done, follow the process in Project creation section.

5.1.2 Enable multikey encryption

1. If not already done, enable the secure boot flow for your application
2. Create 3 AES encryption keys in your application directory

```
edgeprotecttools create-key --key-type AES128 --output keys/aes_key_1.bin
edgeprotecttools create-key --key-type AES128 --output keys/aes_key_2.bin
edgeprotecttools create-key --key-type AES128 --output keys/aes_key_3.bin
```

3. Generate the key encryption key (KEK) using the following command

```
edgeprotecttools create-key --key-type ECDSA-P256 --output keys/enc-ec256-
priv.pem keys/enc-ec256-pub.pem
```

4. Convert the PEM private key to the DER-PKCS8 format using the following command

```
edgeprotecttools convert-key -k keys/enc-ec256-priv.pem -o keys/enc-ec256-
priv.bin -f DER-PKCS8
```

5. Program the private portion of the KEK to the device using openocd at the start address of the “kek_region” defined in the memory map. Check your RRAM memory map in Device Configurator for the correct location of the **kek_region**. The default configuration places the **kek_region** in **0x32039000**

```
<openocd-path>/openocd.exe -s scripts -f interface/kitprog3.cfg -f
target/infineon/pse84xgxs2.cfg -c "init; reset init; flash write_image
keys/enc-ec256-priv.bin 0x32039000; reset; shutdown"
```

6. Open the **Device Configurator** and configure the encryption mode to multi-key encryption in the bootloader solution

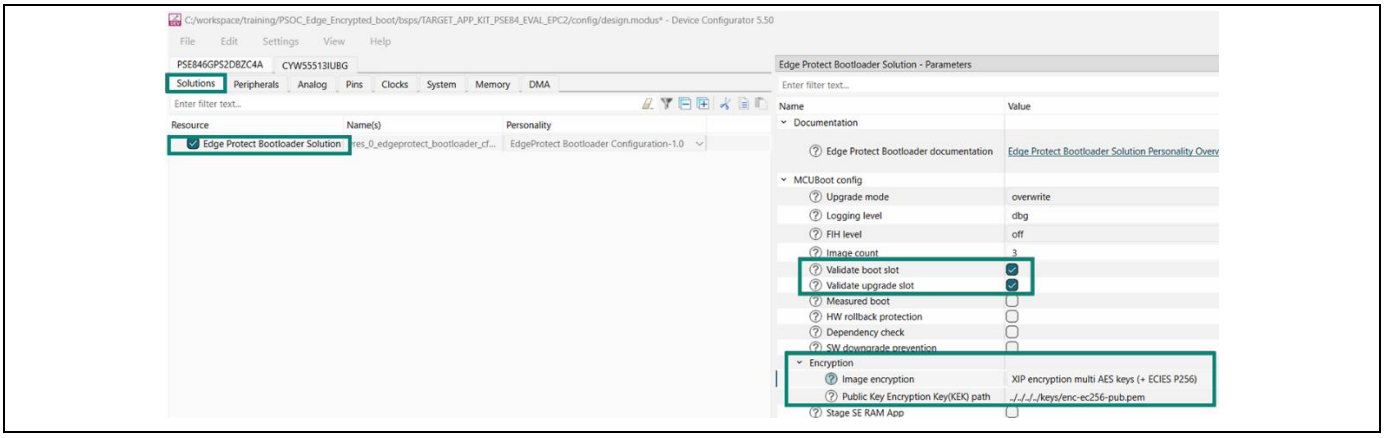


Figure 20 Enable multi-key encryption

7. Set the AES key path for each image

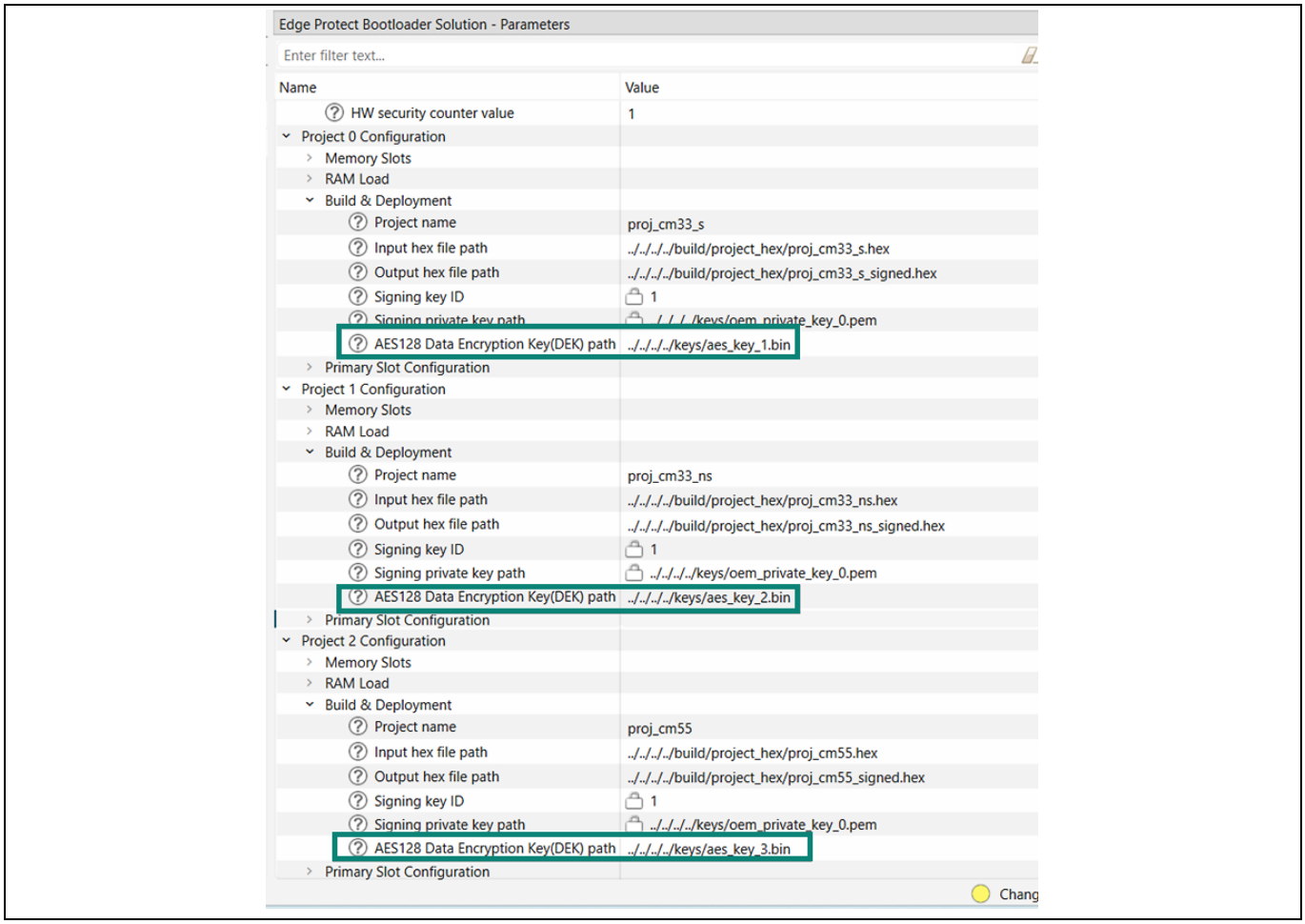
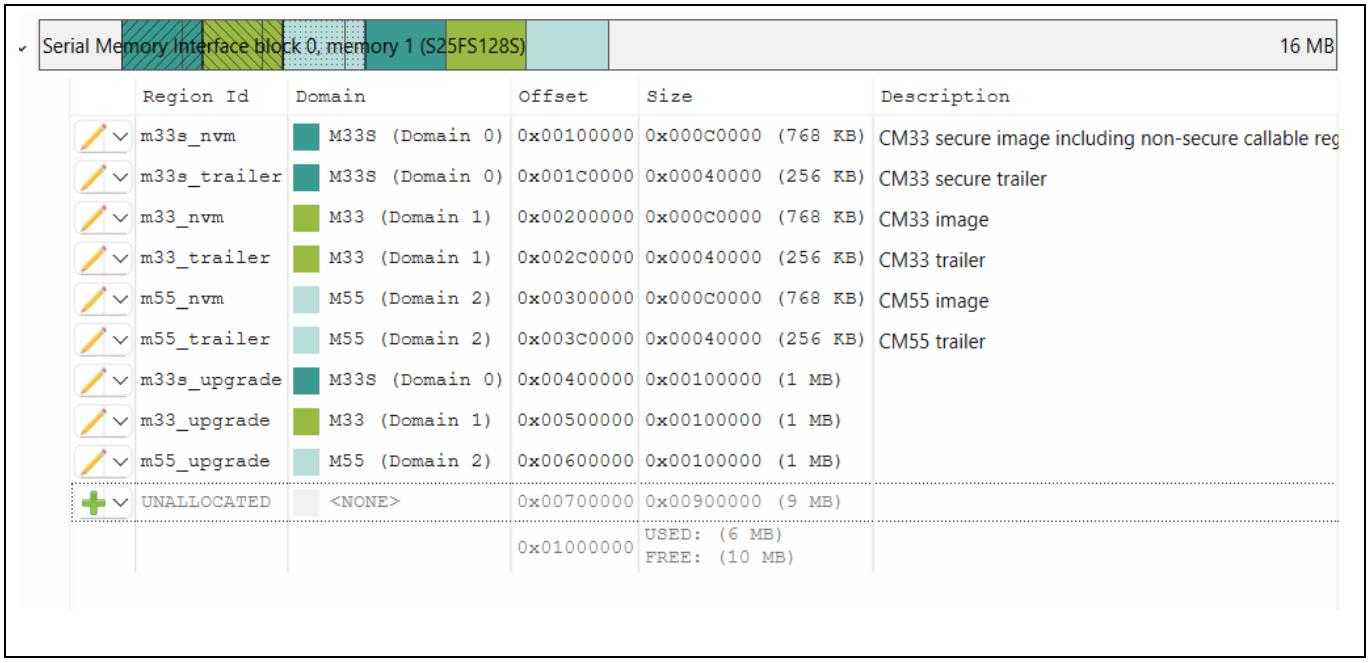


Figure 21 Set AES encryption key

8. Multi-key XIP encryption has strict requirements for encryption region size and start address. The size must be a perfect power-of-two number. The start address must be a multiple of the size number: start address = N * size. To meet these requirements, the project's memory regions must be updated. In the example below all images are sized 0x100000



Region Id	Domain	Offset	Size	Description
m33s_nvm	M33S (Domain 0)	0x00100000	0x000C0000 (768 KB)	CM33 secure image including non-secure callable reg
m33s_trailer	M33S (Domain 0)	0x001C0000	0x00040000 (256 KB)	CM33 secure trailer
m33_nvm	M33 (Domain 1)	0x00200000	0x000C0000 (768 KB)	CM33 image
m33_trailer	M33 (Domain 1)	0x002C0000	0x00040000 (256 KB)	CM33 trailer
m55_nvm	M55 (Domain 2)	0x00300000	0x000C0000 (768 KB)	CM55 image
m55_trailer	M55 (Domain 2)	0x003C0000	0x00040000 (256 KB)	CM55 trailer
m33s_upgrade	M33S (Domain 0)	0x00400000	0x00100000 (1 MB)	
m33_upgrade	M33 (Domain 1)	0x00500000	0x00100000 (1 MB)	
m55_upgrade	M55 (Domain 2)	0x00600000	0x00100000 (1 MB)	
UNALLOCATED	<NONE>	0x00700000	0x00900000 (9 MB)	
		0x01000000	USED: (6 MB) FREE: (10 MB)	

Figure 22 Sample memory regions for multi-key encryption

9. Save your changes in the device configurator
10. Build and program the Encrypted boot application

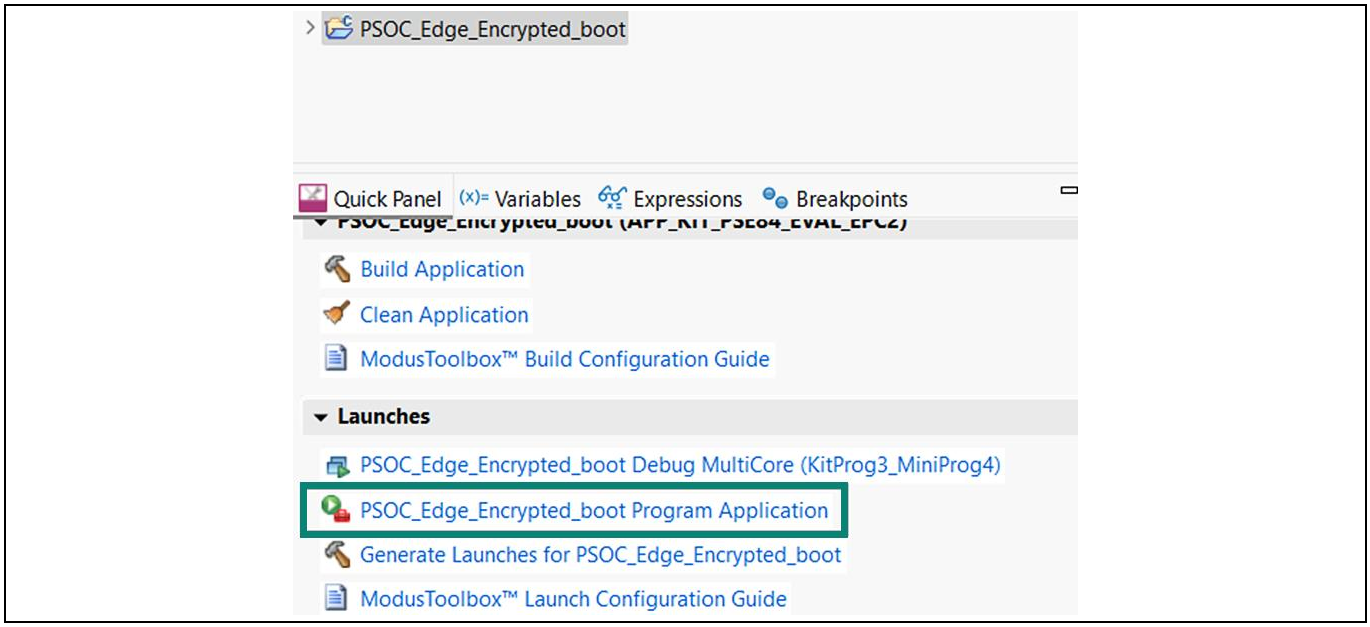


Figure 23 Program application

5.2 Conclusion

We successfully tested the multi-key encryption using the Encrypted Boot code example, and learned:

- How to create the Encrypted Boot code example
- How to add a bootloader to the code example and enable secure boot
- How to program and boot to the encrypted application

Revision history

Document revision	Date	Description of changes
**	2026-01-14	Initial release
*A	2026-05-04	Updated version of tools. Updated screenshots of labs. Updated figures to align with brand guidelines.
*B	2026-06-15	Added KIT_PSE84_AI support in hands-on labs.

Trademarks
All referenced product or service names and trademarks are the property of their respective owners.
PSOC™, formerly known as PSoC™, is a trademark of Infineon Technologies. Any references to PSoC™ in this document or others shall be deemed to refer to PSOC™.

Edition 2026-06-15
Published by

Infineon Technologies AG
Am Campeon 1-15
85579 Neubiberg
Germany

© 2026 Infineon Technologies AG. All Rights Reserved.

Do you have a question about this document?

Email:

erratum@infineon.com

Important Notice

Products which may also include samples and may be comprised of hardware or software or both ("Product(s)") are sold or provided and delivered by Infineon Technologies AG and its affiliates ("Infineon") subject to the terms and conditions of the frame supply contract or other written agreement(s) executed by a customer and Infineon or, in the absence of the foregoing, the applicable Sales Conditions of Infineon. General terms and conditions of a customer or deviations from applicable Sales Conditions of Infineon shall only be binding for Infineon if and to the extent Infineon has given its express written consent.

For the avoidance of doubt, Infineon disclaims all warranties of non-infringement of third-party rights and implied warranties such as warranties of fitness for a specific use/purpose or merchantability.

Infineon shall not be responsible for any information with respect to samples, the application or customer's specific use of any Product or for any examples or typical values given in this document.

If this document is marked as 'Preliminary', 'Target', 'Draft' or 'Proposal', Infineon reserves the right to change all information given in this document at any time without notice. Subject to the development and release of a Product for series supply by Infineon, the technical specifications of the Product are set forth in the relevant datasheet provided by Infineon without such marking.

The data contained in this document is exclusively intended for technically qualified and skilled customer representatives. It is the responsibility of the customer to evaluate the suitability of the Product for the intended application and the customer's specific use and to verify all relevant technical data contained in this document in the intended application and the customer's specific use. The customer is responsible for properly designing, programming, and testing the functionality and safety of the intended application, as well as complying with any legal requirements related to its use.

Unless otherwise explicitly approved by Infineon, Products may not be used in any application where a failure of the Products or any consequences of the use thereof can reasonably be expected to result in personal injury. However, the foregoing shall not prevent the customer from using any Product in such fields of use that Infineon has explicitly designed and sold it for, provided that the overall responsibility for the application lies with the customer.

Infineon expressly reserves the right to use its content for commercial text and data mining (TDM) according to applicable laws, e.g. Section 44b of the German Copyright Act (UrhG).

If the Product includes security features:
Because no computing device can be absolutely secure, and despite security measures implemented in the Product, Infineon does not guarantee that the Product will be free from intrusion, data theft or loss, or other breaches ("Security Breaches"), and Infineon shall have no liability arising out of any Security Breaches.

If this document includes or references software:
The software is owned by Infineon under the intellectual property laws and treaties of the United States, Germany, and other countries worldwide. All rights reserved.

Therefore, you may use the software only as provided in the software license agreement accompanying the software.

If no software license agreement applies, Infineon hereby grants you a personal, non-exclusive, non-transferable license (without the right to sublicense) under its intellectual property rights in the software (a) for software provided in source code form, to modify and reproduce the software solely for use with Infineon hardware products, only internally within your organization, and (b) to distribute the software in binary code form externally to end users, solely for use on Infineon hardware products. Any other use, reproduction, modification, translation, or compilation of the software is prohibited.

For further information on the Product, technology, delivery terms and conditions, and prices, please contact your nearest Infineon office or visit https://www.infineon.com.